

GitLab

Administrator's Guide

Beginner to Practical Administration



INSTALL
CONFIGURE



BACKUP
RESTORE



MANAGE
USERS



CI/CD
RUNNERS



SECURE



GROUPS
PROJECTS



MONITOR
TROUBLESHOOT

01

Understanding GitLab Administration



What is GitLab?



GitLab is a complete DevOps platform that allows teams to manage source code, CI/CD pipelines, security, packages and project collaboration in one place.

Why GitLab Administration?

- Keeps the platform running and secure
- Manages users, groups and permissions
- Controls storage, backups and upgrades
- Configures CI/CD and runners
- Troubleshoots issues
- Ensures high availability and performance
- Supports developer productivity

What Does a GitLab Administrator Manage?



Users & Access

Create, manage and control user accounts, roles and permissions.



Groups & Projects

Organize teams, namespaces and project settings.



CI/CD

Configure pipelines, variables, runners and job execution.



Registries

Manage container registry, package registry and storage settings.



Security

Manage authentication, SSH keys, tokens, SSO, 2FA and security policies.



Storage & Backup

Monitor disk usage, manage repositories, artifacts, logs and backups.



Maintenance

Perform updates, patches and system maintenance.



Monitoring & Troubleshooting

Monitor services, check logs, identify and resolve issues.

Who Uses GitLab?



Developers
Write and push code



Engineering Teams
Collaborate on projects



DevOps Engineers
Build and deploy applications



GitLab Administrators
Manage and operate the platform



Key Takeaway

A GitLab administrator keeps the platform secure, stable, available and productive for engineering teams.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

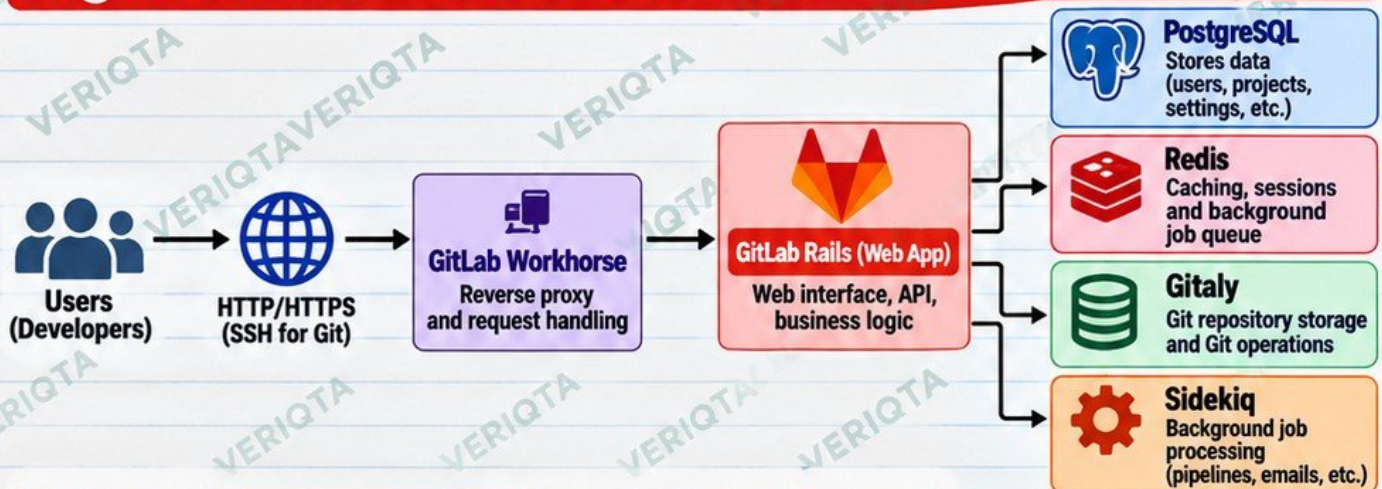
What is GitLab Architecture?

GitLab is made up of several services that work together to provide a complete DevOps platform for source code management, CI/CD, security, packages and collaboration.

Why Multiple Components?

- Each component handles a specific job
- Improves performance and reliability
- Allows background tasks, storage and web requests to be separated
- Helps GitLab scale and stay stable in production

High-Level GitLab Architecture



GitLab Workhorse

- Reverse proxy between users and GitLab Rails
- Handles large file uploads
- Optimizes Git operations
- Improves security and performance

PostgreSQL

- Stores all GitLab data
- Users, groups, projects, issues, MRs
- Configuration and application data
- Requires regular backups

Gitaly

- Stores Git repositories (all project code)
- Handles Git operations (clone, fetch, push)
- Keeps repositories secure and isolated
- Supports high availability and scaling

GitLab Rails (Web Application)

- Main application (written in Ruby on Rails)
- Handles the web interface and API
- Manages users, projects and settings
- Triggers background jobs for heavy tasks

Redis

- In-memory data store
- Caching for faster performance
- Stores user sessions
- Used as a queue for background jobs with Sidekiq

Sidekiq

- Processes background jobs
- Handles CI/CD tasks, emails, webhooks
- Runs asynchronously so the web interface stays fast
- Uses Redis as the job queue

How They Work Together

- 1 Users access GitLab through HTTPS or SSH.
- 2 GitLab Workhorse handles incoming requests.
- 3 GitLab Rails processes web requests and API calls.
- 4 PostgreSQL stores and retrieves data.
- 5 Redis caches data and queues background jobs.
- 6 Sidekiq processes background tasks (pipelines, emails, etc.).
- 7 Gitaly manages all Git repository storage and Git operations.

Key Takeaway

GitLab is a collection of specialized components. Each component plays a specific role, and together they provide a secure, scalable and reliable DevOps platform.

Real-World Analogy



Think of GitLab like a company: Rails is the office, PostgreSQL is the filing system, Redis is the fast-notes system, Gitaly is the storage warehouse, Sidekiq is the background team, and Workhorse is the receptionist.

03

GitLab Deployment Options



GitLab.com versus Self-Managed, installation methods and deployment decisions.

GitLab.com (SaaS)



- ✓ Managed by GitLab
- ✓ No installation required
- ✓ Automatic updates
- ✓ Highly available
- ✓ Access from anywhere
- ✓ Ideal for small to medium teams

Self-Managed GitLab



- ✓ You install and manage it
- ✓ Full control and customization
- ✓ Deploy on your own servers or cloud (AWS, Azure, etc.)
- ✓ You handle updates, backups and security
- ✓ Ideal for large teams or organizations with strict compliance requirements

Key Differences

Feature	GitLab.com (SaaS)	Self-Managed GitLab
Hosting	GitLab hosts and manages	You host and manage
Installation	Not required	Required
Updates	Automatic	Manual (you plan and perform)
Customization	Limited (based on plan)	Full control
Security & Compliance	Shared responsibility	Your responsibility
Cost	Subscription based	Infrastructure + maintenance cost
Best For	Individuals and small/medium teams	Large teams, regulated industries, and organizations needing full control

Self-Managed Installation Methods

Omnibus Package



- Official and recommended
- Easiest way to install
- Includes all components
- Runs on Linux (Debian/Ubuntu, RHEL/CentOS, etc.)

Docker



- Quick installation
- Good for testing and labs
- Uses Docker containers
- Not recommended for large production environments

Kubernetes (Helm)



- Deploy using Helm charts
- Good for scalable environments
- Requires Kubernetes knowledge
- Suitable for production in cloud or on-prem clusters

Source Installation



- Manual installation
- More complex
- Used for advanced customization
- Not commonly used for most environments

When to Choose Each Option?

- ✓ Choose **GitLab.com** when you want a fast, managed solution with minimal maintenance.
- ✓ Choose **Self-Managed** when you need full control, compliance, customization or on-premises deployment.
- ✓ Choose **Docker** or **Kubernetes** when you want container-based deployment, scalability or a cloud-native setup.

Key Takeaway



Both options provide the same core features. The right choice depends on your team size, compliance requirements, budget and how much control you need over the platform.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

03

Installation prerequisites, package installation and initial configuration.

1. Prerequisites



A Linux server (Ubuntu or RHEL/CentOS)



Minimum 4 CPU cores
8 GB RAM (16 GB recommended)



At least 100 GB disk space
(SSD recommended)



A domain name (e.g. gitlab.example.com)



Firewall configured (HTTP/HTTPS/SSH)



A user with sudo or root access

2. Update the System



Ubuntu / Debian

```
sudo apt update
sudo apt upgrade -y
```

RHEL / CentOS (Rocky/AlmaLinux)

```
sudo dnf update -y
```



Tip: Always start with an updated system to avoid dependency issues.

3. Install Required Packages



Ubuntu / Debian

```
sudo apt install -y curl openssh-server
ca-certificates tzdata
```



RHEL / CentOS (Rocky/AlmaLinux)

```
sudo dnf install -y curl openssh-server
ca-certificates tzdata policycoreutils
```

4. Add GitLab Repository



Ubuntu / Debian

```
curl https://packages.gitlab.com/install/
repositories/gitlab/gitlab-ee/script.deb.sh
| sudo bash
```



RHEL / CentOS

```
curl https://packages.gitlab.com/install/
repositories/gitlab/gitlab-ee/script.rpm.sh
| sudo bash
```

5. Install GitLab



Ubuntu / Debian

```
sudo apt install gitlab-ee -y
```



RHEL / CentOS

```
sudo dnf install gitlab-ee -y
```

6. Set the External URL



```
sudo nano /etc/gitlab/gitlab.rb
```

Set or update this line:

```
external_url 'http://gitlab.example.com'
```



Use your real domain name and prefer HTTPS in production.

7. Reconfigure GitLab



```
sudo gitlab-ctl reconfigure
```



This may take several minutes on the first run.

8. Access GitLab



Open your browser and visit:

```
http://gitlab.example.com
```

or <https://gitlab.example.com>

9. Verify GitLab is Running



```
sudo gitlab-ctl status
```



All services
should show: **run**

Important Notes

- The initial root password is in: `/etc/gitlab/initial_root_password`
- Use HTTPS with a valid SSL certificate in production
- Configure backups after installation
- Adjust resources based on your team size and usage



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

04

Understanding /etc/gitlab/gitlab.rb, reconfigure and service management.

1. What is gitlab.rb?



- /etc/gitlab/gitlab.rb is the main configuration file for a self-managed GitLab instance.
- It controls all GitLab components and settings.
- You change values in this file, then run reconfigure to apply them.

2. Common Configuration Areas



- External URL (domain or IP)
- HTTP/HTTPS and SSL settings
- Email settings (SMTP)
- GitLab services and ports
- Registry settings
- Backup settings
- Resource limits (memory, PostgreSQL, etc.)

3. View and Edit the Configuration File



View the file

```
sudo cat /etc/gitlab/gitlab.rb
```

Edit the file

```
sudo nano /etc/gitlab/gitlab.rb
```

4. Key Configuration Examples

External URL for your GitLab instance

```
external_url 'https://gitlab.example.com'
```

Enable and configure SSL (example with Let's Encrypt)

```
letsencrypt['enable'] = true
```

Configure SMTP for email notifications

```
gitlab_rails['smtp_enable'] = true
```

```
gitlab_rails['smtp_address'] = 'smtp.example.com'
```

```
gitlab_rails['smtp_port'] = 587
```

```
gitlab_rails['smtp_user_name'] = 'gitlab@example.com'
```

```
gitlab_rails['smtp_password'] = 'your_password'
```

```
gitlab_rails['smtp_domain'] = 'example.com'
```

Registry settings (optional)

```
registry_external_url 'https://registry.example.com'
```

5. Apply Changes with Reconfigure



```
sudo gitlab-ctl reconfigure
```



- This command reads /etc/gitlab/gitlab.rb and applies your changes to all GitLab components. It may take several minutes.

6. Common Service Management Commands

Action	Command
Check all services	<code>sudo gitlab-ctl status</code>
Start all services	<code>sudo gitlab-ctl start</code>
Stop all services	<code>sudo gitlab-ctl stop</code>
Restart all services	<code>sudo gitlab-ctl restart</code>
Restart a specific service	<code>sudo gitlab-ctl restart <service></code>
Show service logs	<code>sudo gitlab-ctl tail <service></code>
Reconfigure GitLab	<code>sudo gitlab-ctl reconfigure</code>

7. Verify the Configuration



```
sudo gitlab-ctl status
```

- All services should show: **run**

```
sudo gitlab-ctl show-config
```

- Shows the final applied configuration.

```
curl -I https://gitlab.example.com
```

- Verify GitLab is accessible.

8. Important Tips



- Always make a backup of gitlab.rb before changes.
- Use a real domain name and HTTPS in production.
- Reconfigure can take a few minutes.
- Check service status if something doesn't work.
- Keep SSH access in case you lose web access.

Creating, managing, blocking and deactivating users.

1. User Types in GitLab

-  **Regular User** Can create projects and collaborate based on permissions.
-  **Administrator** Full access to user, group, project and instance settings.
-  **External User** Can be added to specific projects or groups with limited access.
-  **Bot User** Used for automation (e.g. CI/CD, integrations).

3. Create a User Using Command Line

```

sudo gitlab-rails console
user = User.new(
  email: 'user@example.com',
  username: 'newuser',
  name: 'New User',
  password: 'StrongPassword123!'
)
user.save!
  
```

- This method is useful for automation or bulk user creation.
- You can also set `user['admin'] = true` for an admin user (if required).

5. Block a User

-  • Blocking a user prevents them from signing in.
- Their projects and data remain intact.
- They can be unblocked at any time.

```

sudo gitlab-rails console
user = User.find_by(username: 'username')
user.block
user.save!
  
```

7. Reset a User's Password

-  • You can reset a user's password from the UI or using the command line.

```

sudo gitlab-rails console
user = User.find_by(username: 'username')
user.password = 'NewSecurePassword123!'
user.save!
  
```

2. Create a New User (Web UI)

- 1 Sign in as an administrator.
- 2 Go to Admin Area → Users.
- 3 Click New user.
- 4 Enter name, username, email and password.
- 5 Choose the appropriate role (usually Regular User).
- 6 Click Create user.



The screenshot shows the GitLab Admin Area with the 'Users' tab selected. On the right, the 'New user' form is visible with fields for Name, Username, Email, Password, and Role (set to Regular User). A 'Create user' button is at the bottom right.

4. Manage Existing Users

-  • View user details
 - Edit user information
 - Reset password
 - Change the user's role
 - View user activity
 - Search and filter users
- 
- The screenshot shows the sidebar menu in the GitLab Admin Area with the 'Users' tab selected. The menu includes options for Admin Area, Overview, Users, Groups, Projects, Roles and permissions, and Settings.

6. Deactivate a User

-  • Deactivating a user disables their account.
- They cannot sign in.
- Their data (projects, commits, issues, etc.) is preserved.
- Use this for offboarding or when a user no longer needs access.

```

sudo gitlab-rails console
user = User.find_by(username: 'username')
user.deactivate
user.save!
  
```

8. Important Notes

-  • Admin users should be limited and carefully managed.
- Use strong passwords and enforce 2FA.
- Regularly review active users.
- Deactivate or block users who no longer need access.
- Audit user activities for security and compliance.
- Use groups instead of giving direct project access where possible.

Organizing GitLab resources for engineering teams.

1. Why Organize GitLab Resources?



GitLab

- Keeps projects structured and easy to manage.
- Improves collaboration across teams.
- Controls access and permissions.
- Simplifies CI/CD, security and compliance.
- Scales as your organization grows.

2. Key Concepts

Groups



Top level container for projects and subgroups. Used to organize teams and resources.

Projects



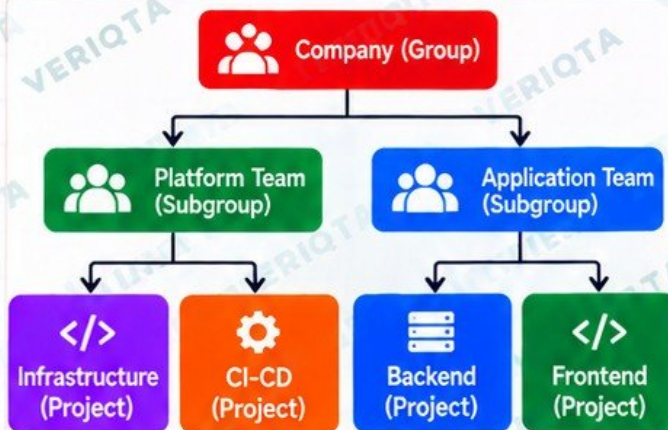
A repository with its own settings, CI/CD, issues and merge requests.

Namespaces



The scope for groups and projects. Defines the path and ownership (e.g. group/subgroup/project).

3. Group Structure Example



4. Useful GitLab Commands (Group and Project)

```

>_ # Create a group
curl --header "PRIVATE-TOKEN: YOUR_TOKEN" \
--data "name=platform-team&path=platform" \
https://gitlab.example.com/api/v4/groups

# Create a subgroup
curl --header "PRIVATE-TOKEN: YOUR_TOKEN" \
--data "name=devops&path=devops" \
https://gitlab.example.com/api/v4/groups/1/groups

# Create a project in a group
curl --header "PRIVATE-TOKEN: YOUR_TOKEN" \
--data "name=infrastructure&namespace_id=1" \
https://gitlab.example.com/api/v4/projects
  
```

5. Access and Permissions

Role	Purpose	Key Capabilities
Owner	Full control	Manage group, projects, members and settings.
Maintainer	Manage resources	Push code, manage CI/CD, settings and members.
Developer	Contribute to projects	Push code, create merge requests.
Reporter	Read and test	View and access project resources.
Guest	Limited access	Minimal read-only access.

6. Common API Endpoints

Endpoint	Purpose
GET /groups	List all groups
POST /groups	Create a new group
GET /groups/:id	Get group details
PUT /groups/:id	Update group settings
GET /groups/:id/projects	List projects in a group
POST /projects	Create a new project
GET /projects/:id	Get project details
PUT /projects/:id	Update project settings
DELETE /projects/:id	Delete a project

7. Common Mistakes

- Not using groups and subgroups.
- Flat structure with too many projects.
- Incorrect permissions and access levels.
- Not following naming conventions.
- Not separating environments (e.g. dev, staging, prod).

8. Best Practices

- ✓ Use groups and subgroups to match your organization structure.
- ✓ Set clear naming conventions.
- ✓ Apply least privilege access.
- ✓ Use group level CI/CD and security policies.
- ✓ Keep projects organized by team, product or environment.

9. Your Turn (Practice)

- 1 Create a group and a subgroup.
- 2 Add a project to the subgroup.
- 3 Configure different access levels for team members.
- 4 Explore group and project settings.
- 5 Use the GitLab API to list all groups and projects.

Understanding access levels, protected resources and least privilege.

1. Why Roles and Permissions are Important?

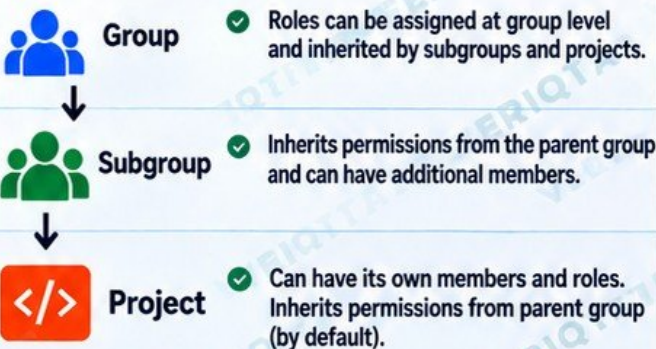


- Controls who can access and make changes to your resources.
- Protects sensitive code, secrets and infrastructure.
- Reduces the risk of accidental or malicious changes.
- Enforces least privilege access.
- Helps with compliance and audit requirements.

2. GitLab Access Levels

Role	Level	Key Capabilities
Owner	Highest	Full control. Manage project settings, members, CI/CD, and delete project.
Maintainer	High	Manage repository, CI/CD, issues, merge requests and settings.
Developer	Medium	Push code, create branches, create merge requests and manage pipelines.
Reporter	Low	Read code, view issues, view pipelines and download artifacts.
Guest	Lowest	Limited read access to project resources.

3. Role Scope and Inheritance



4. Assign Roles (UI and API)

Using GitLab UI



- Go to Project / Group > Members.
- Click Invite members.
- Select a role (Guest, Reporter, Developer, Maintainer or Owner).
- Set an expiration date (optional).
- Click Add to project/group.

Using GitLab API

```
# Add a member to a project
curl --request POST \
  --header "PRIVATE-TOKEN: \
  YOUR_TOKEN" \
  --data "user_id=123
  &access_level=30" \
  "https://gitlab.example.com/
  api/v4/projects/1/members"
```

Access level ID:
10=Guest 20=Reporter 30=Developer
40=Maintainer 50=Owner

5. Protected Resources

- Protected Branches** ✓ Restrict who can push or merge.
✓ Require merge requests and approvals.
✓ Protect main or release branches.
- Protected Tags** ✓ Restrict who can create or update tags.
✓ Used for release management.
✓ Prevent accidental or unauthorized releases.
- Protected Environments** ✓ Restrict deployment access.
✓ Require approvals for production.
✓ Control who can deploy to specific environments (e.g. staging, production).

6. Example Protected Branch Rule (UI)

Protect branch: main

- ☒ Protect this branch
 - ☒ Require merge requests
 - ☒ Require approvals Approvals required:
 - ☒ Restrict who can push Maintainers
 - ☒ Restrict who can merge Maintainers
- Also allow: ☐ Developers ☐ Maintainers

7. Least Privilege and Best Practices



- Give only the minimum access required.
- Use groups to manage large teams.
- Regularly review members and access levels.
- Use protected branches and environments.
- Remove unused or inactive accounts.
- Use group level permissions for consistency.

8. Audit and Review Access



- Review project and group members regularly.
- Use audit events to track permission changes.
- Identify unused or excessive permissions.
- Remove inactive users and external access.
- Monitor access to protected resources.
- Keep a record for compliance and security.

9. Common Mistakes

- Giving everyone maintainer access.
- Not using groups for permission management.
- Not protecting critical branches.
- Not reviewing access regularly.
- Leaving unused accounts with access.

10. Useful Commands (View Members)

```
# List project members
curl --header "PRIVATE-TOKEN: YOUR_TOKEN" \
  "https://gitlab.example.com/api/v4/
  projects/1/members"
# List group members
curl --header "PRIVATE-TOKEN: YOUR_TOKEN" \
  "https://gitlab.example.com/api/v4/
  groups/1/members"
```

11. Your Turn (Practice)

- Create a group and a project.
- Add users with different roles.
- Protect the main branch.
- Set merge request approvals.
- Review and audit project members.
- Apply least privilege access.

SSH keys, personal access tokens, two-factor authentication and SSO concepts.

1. What is Authentication and Access Security?

Authentication verifies **who you are**.
Authorization controls **what you can do**.
Together, they protect your systems, data and cloud resources from unauthorized access.

2. Why It Matters in DevOps

- Prevents unauthorized access to servers, repositories and cloud resources.
- Protects sensitive data and infrastructure.
- Reduces the risk of credential theft.
- Ensures only the right people have access.
- Required for compliance and security best practices.

3. How It Works (High Level)



4. SSH Keys (Secure Access to Servers)

- SSH keys are a secure way to connect to Linux servers without a password.
- You create a key pair: a private key (kept secret) and a public key (added to the server).
- Used by DevOps engineers to access servers, VMs and cloud instances.

```
# Create a new SSH key pair
ssh-keygen -t ed25519 -C "your_email@example.com"

# Copy public key to server
ssh-copy-id user@server-ip
```

5. Personal Access Tokens (PAT)

- A PAT is a long, randomly generated token used instead of a password.
- Used for GitHub, GitLab, cloud providers (AWS, Azure, GCP) and APIs.
- Useful for automation, CI/CD and tools.
- Treat it like a password and keep it secret.

```
# Example: Use PAT with GitHub
git clone https://<username>:<PAT>@github.com/your-repo.git
```

6. Two-Factor Authentication (2FA)

- Adds a second layer of security.
- Even if your password is stolen, attackers cannot login without the second factor.
- Common 2FA methods:
 - Authenticator app (Google Authenticator)
 - SMS code
 - Hardware security key (e.g. YubiKey)



7. Single Sign-On (SSO)

- SSO allows you to login once and access multiple tools and applications.
- Common with company accounts using identity providers (IdP) like:
 - Microsoft Entra ID (Azure AD)
 - Okta
 - Google Workspace
 - IAM (AWS)



8. Common Mistakes

- Sharing private SSH keys or PATs.
- Storing tokens in code or public repos.
- Not enabling 2FA.
- Using weak passwords.
- Giving excessive permissions.

9. Security Best Practices

- Use SSH keys instead of passwords.
- Enable 2FA on all accounts.
- Use PATs with minimal permissions.
- Rotate tokens regularly.
- Use SSO for company resources.
- Apply least privilege access.
- Never store secrets in code repositories.

10. Your Turn (Practice)

- 1 Create an SSH key pair on your machine.
- 2 Add the public key to a test server.
- 3 Enable 2FA on your GitHub account.
- 4 Create a personal access token.
- 5 Explore SSO with a free account (e.g. Google Workspace or Okta demo).

10 GitLab Repository Administration

Repository settings, protected branches, merge requests and push rules.

1. What is GitLab Repository Administration?



- It is the process of managing a GitLab project to control how code is stored, accessed and changed.
- You configure settings, permissions, branch rules and policies.
- It helps keep your code secure, organized and ready for team collaboration.

2. Key Components



Repository settings

Control project visibility, permissions and features.



Protected branches

Restrict who can push or merge to important branches.



Merge requests

Review and approve code changes before merging.



Push rules

Enforce rules to keep your repository clean and secure.

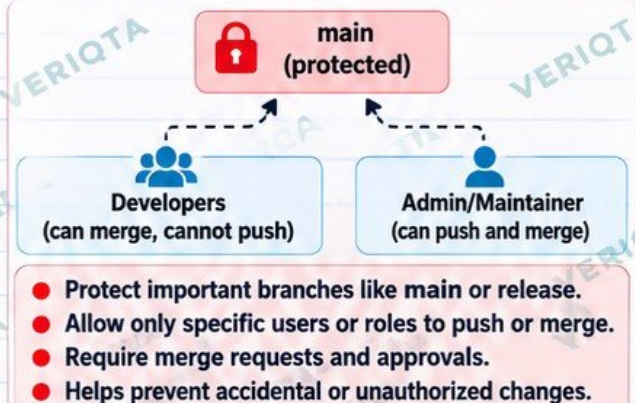
3. Repository Settings (Project Settings)

Project

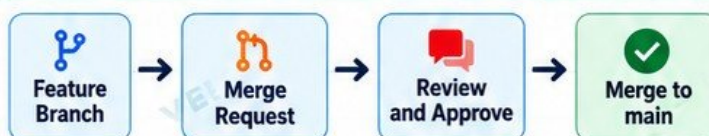
- General
- Integrations
- Webhooks
- Access Tokens
- Repository
- Merge requests
- CI/CD
- Packages and registries

- Go to Settings in your GitLab project.
- Configure project name, description and visibility (public, internal or private).
- Manage members and their roles (Guest, Reporter, Developer, Maintainer, Owner).
- Enable or disable features such as issues, merge requests and wiki.

4. Protected Branches



5. Merge Requests (Code Review and Collaboration)



- A merge request (MR) is a request to merge code changes from one branch into another.
- Team members review the changes, leave comments and approve them.
- CI/CD pipelines can run automatically to test the changes.
- Once approved, the code can be merged.

6. Push Rules



Push rules enforce rules when developers push code to the repository. They help maintain code quality and security.

Common push rules

- Block large files.
- Prevent sensitive data (e.g. API keys, passwords).
- Enforce commit message format.
- Allow only specific file types.
- Block force pushes to certain branches.

7. Example: Protect the main Branch (UI)

Protected branches

Branch name	Allowed to push	Allowed to merge
main	Maintainers only	Maintainers

☒ Require merge requests

8. Example: Push Rule (Block Large Files)

Maximum file size ?

10 MB

Reject files larger than this size when pushing to the repository.

Common Mistakes

- Not protecting the main branch.
- Allowing everyone to push directly to main.
- Not enabling merge request approvals.
- Not setting push rules.
- Giving too many permissions to users.

10. Best Practices

- Protect main and important branches.
- Use merge requests for all changes.
- Require approvals and passing pipelines.
- Set push rules to block large files and sensitive data.
- Follow the principle of least privilege.
- Regularly review project members and permissions.

Your Turn (Practice)

- Create a new GitLab project.
- Configure project settings (private).
- Protect the main branch.
- Enable merge request approvals.
- Add a push rule to block files larger than 10 MB.

11 GitLab CI/CD Administration

Pipelines, .gitlab-ci.yml, variables and instance-level settings.

1. What is GitLab CI/CD?



- GitLab CI/CD automatically builds, tests and deploys your code.
- It runs pipelines based on rules you define in .gitlab-ci.yml.
- It helps teams deliver software faster and with fewer mistakes.

2. Key CI/CD Components



Pipeline

Runs a series of jobs from your .gitlab-ci.yml file.



Runner

Executes the jobs defined in the pipeline.



.gitlab-ci.yml

Configuration file that defines stages and jobs.



Variables

Store sensitive or dynamic values for your pipeline.



Instance-level settings

Global CI/CD settings managed by administrators.

3. How GitLab CI/CD Works



- When you push code, GitLab reads your .gitlab-ci.yml file.
- It creates a pipeline and runs the defined jobs using runners.
- If all jobs succeed, the pipeline can deploy your application.

4. Example .gitlab-ci.yml (Simple Pipeline)

```
.gitlab-ci.yml

stages:
  - build
  - test
  - deploy

build-job:
  stage: build
  script:
    - echo "Building application..."

test-job:
  stage: test
  script:
    - echo "Running tests..."

deploy-job:
  stage: deploy
  script:
    - echo "Deploying application..."
```

5. CI/CD Variables

- Variables store values used in your pipeline.
- Use them for sensitive data such as API keys, passwords, tokens and cloud credentials.
- You can set variables at project, group or instance level.
- Mark sensitive variables as Protected and Masked.

Example Project Variable (UI)

Key

Value

AWS_ACCESS_KEY_ID

.....



Protected



Masked

6. Instance-Level CI/CD Settings (Admin)



- Managed by GitLab administrators.
- Configure runners, default CI/CD settings, and pipeline limits.
- Control which features are allowed or disabled.

7. Common Use Cases

- ✓ Build and test applications automatically.
- ✓ Deploy to staging and production.
- ✓ Use different variables per environment.
- ✓ Run security scans (SAST, dependency scan).
- ✓ Deploy using Docker, Kubernetes or cloud providers.

Common Mistakes

- Not defining stages correctly.
- Hardcoding secrets in .gitlab-ci.yml.
- Not using protected variables.
- Using shared runners for sensitive jobs.
- Pipelines failing due to missing variables.
- Giving too many permissions to runners.

9. Best Practices

- ✓ Use variables instead of hardcoded values.
- ✓ Protect and mask sensitive variables.
- ✓ Use protected branches for deployments.
- ✓ Use specific runners for sensitive jobs.
- ✓ Keep .gitlab-ci.yml in version control.
- ✓ Set pipeline rules to run only when needed.
- ✓ Monitor pipeline logs and artifacts.

10. Your Turn (Practice)

- 1 Create a new GitLab project.
- 3 Add the .gitlab-ci.yml file from above.
- 3 Set a project variable (e.g. API key).
- 4 Run the pipeline and check the jobs.
- 5 Try modifying a variable and run again.

12 GitLab Runner Administration

Runner installation, registration, executors and runner management.

1. What is GitLab Runner?



- GitLab Runner is an agent that runs CI/CD jobs defined in .gitlab-ci.yml.
- It picks up jobs from GitLab, executes them and returns the results.
- Runners can run on your own servers, VMs, containers or in the cloud.

2. How GitLab Runner Works



- ✓ Runners continuously check GitLab for new jobs.
- ✓ They execute the jobs using a configured executor.
- ✓ The job logs and status are sent back to GitLab.

3. Types of Runners

Runner Type	Description	When to Use
Shared Runner	Provided by GitLab instance (multi-tenant)	Small projects or testing
Group Runner	Shared across a group of projects	Team or department level
Project Runner	Used only by a specific project	Dedicated project control
Specific Runner	Assigned to specific projects using tags	Control which jobs run on the runner

4. Executor Types

Executor	Description	Common Use Case
Shell	Runs jobs directly on the host machine	Simple setups, VMs and bare metal
Docker	Runs jobs in Docker containers	Isolated and clean environments
Docker Machine	Creates temporary VMs with Docker	Auto scaling in cloud environments
Kubernetes	Runs jobs in Kubernetes pods	Scalable and modern environments
Custom	Use your own executor	Advanced or special requirements

5. Install GitLab Runner (Linux)

```
> # Download and install
sudo curl -L https://packages.gitlab.com/
gitlab/gitlab-runner/packages/debian/
bookworm/gitlab-runner_amd64.deb -o gitlab-runner.deb
sudo dpkg -i gitlab-runner.deb

# Start and enable service
sudo systemctl enable gitlab-runner
sudo systemctl start gitlab-runner
sudo systemctl status gitlab-runner
```

6. Register a Runner

```
> sudo gitlab-runner register
--url https://gitlab.com/
--registration-token YOUR_TOKEN_HERE
--description "my-runner"
--tag-list "docker,linux"
--executor docker
--docker-image alpine:latest
```

- ✓ Get the registration token from: Project → Settings → CI/CD → Runners
- ✓ You can register a runner as shared, group or project runner depending on where you get the token.

7. Common Runner Management Commands

Command	Purpose
sudo gitlab-runner list	List registered runners
sudo gitlab-runner verify	Check runner connectivity
sudo gitlab-runner unregister	Remove a runner
sudo gitlab-runner restart	Restart the service
sudo gitlab-runner status	Check runner service status

8. Instance-Level Runner Settings (Admin)



- Enable or disable shared runners.
- Set maximum number of concurrent jobs.
- Configure allowed executors.
- Set runner registration requirements.
- Monitor runner usage and performance.
- Define runner token expiration and policies.

Common Mistakes

- Using shared runners for sensitive jobs.
- Using wrong executor for your environment.
- Not tagging runners correctly.
- Exposing runners without proper security.
- Not monitoring disk space on runners.

10. Best Practices

- ✓ Use dedicated runners for production.
- ✓ Use tags to control which jobs run on which runners.
- ✓ Keep runners updated.
- ✓ Use Docker or Kubernetes executors for isolation.
- ✓ Restrict runner registration.
- ✓ Monitor runner health and job success rate.

11. Your Turn (Practice)

- 1 Install GitLab Runner on a test server.
- 2 Register a new runner with Docker executor.
- 3 Add tags to the runner.
- 4 Verify the runner using gitlab-runner verify.
- 5 Run a pipeline and check the job logs.

Registry configuration, storage, permissions and cleanup.

1. What are GitLab Registries?



Container Registry
Stores Docker images for your projects.



Package Registry
Stores other packages such as Maven, npm, PyPI, NuGet and Helm charts.

- Built into GitLab.
- Integrates with your CI/CD pipelines.
- Keeps your images and packages secure.
- Helps teams share and version artifacts for deployments.

2. How It Works (Container Registry Flow)



Developer builds image



Push to GitLab Registry



Images stored in project registry



CI/CD pulls image for deployment

- Images are stored per project by default.
- You authenticate using your GitLab username and token.
- CI/CD pipelines can push and pull images automatically.

3. Registry Types and Use Cases

Registry Type	What It Stores	Common Use Cases
Container Registry	Docker and OCI images	Application images, base images, microservices
Maven Registry	Java packages (Maven)	Java applications, internal libraries
npm Registry	Node.js packages	Frontend and backend JavaScript projects
PyPI Registry	Python packages	Python applications and internal tools
Generic Registry	Any artifact	Binaries, Helm charts, other package types

4. Authenticate and Push a Docker Image

```
> # Login to GitLab Container Registry
docker login registry.gitlab.com
# Username: your GitLab username
# Password: your personal access token (PAT)

# Tag your image
docker tag myapp:latest \
registry.gitlab.com/your-username/myapp:latest

# Push the image
docker push registry.gitlab.com/your-username/myapp:latest
```

5. Use in .gitlab-ci.yml

```
> stages:
  - build
  - push

build-and-push:
  stage: push
  script:
    - docker build -t registry.gitlab.com/$CI_PROJECT_PATH/myapp:$CI_COMMIT_SHA .
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
    - docker push registry.gitlab.com/$CI_PROJECT_PATH/myapp:$CI_COMMIT_SHA
```

6. Permissions and Access Control

Role	Can Do	Use Case
Guest	Pull public images (if allowed)	Read-only access
Reporter	Pull images	View and test images
Developer	Push and pull images	Build and publish images
Maintainer	Manage settings and permissions	Project administration
Owner (Instance)	Configure instance-level registry settings	Administer storage, limits and policies

7. Storage and Cleanup



- Registry storage can grow quickly.
- Use retention policies to remove old images and packages.
- Set expiration rules for tags.
- Clean up unused or untagged images.
- Monitor registry storage usage at project and instance level.

8. Instance-Level Settings (Admin)



- Enable or disable container and package registries.
- Set storage limits and quotas.
- Configure cleanup policies and retention rules.
- Restrict which projects can use registries.
- Configure object storage (e.g. S3) for large environments.
- Monitor usage and performance.

9. Common Mistakes

- Not using a personal access token (PAT).
- Storing images without cleanup.
- Using insecure or public images.
- Giving too many permissions.
- Not setting retention policies.

10. Best Practices

- ✓ Use CI/CD to build and push images.
- ✓ Use protected branches for production images.
- ✓ Use least privilege access.
- ✓ Enable image/tag retention and cleanup.
- ✓ Use secure base images and scan for vulnerabilities.
- ✓ Monitor registry storage and usage regularly.

11. Your Turn (Practice)

- 1 Build a Docker image for a sample application.
- 2 Push the image to your GitLab registry.
- 3 Use the image in a .gitlab-ci.yml pipeline.
- 4 Set a retention policy for old images.
- 5 Explore package registry (e.g. npm or Maven).

Repository storage, Gitaly, artifacts, logs and disk capacity.

1. What is Repository Storage?



GitLab stores your source code, artifacts, container images, logs and other project data on disk.

- Repositories are stored and managed by Gitaly.
- Artifacts, packages, container images and LFS files consume storage.
- Logs and backups also use disk space.
- Proper storage management keeps GitLab stable and fast.

2. Key Storage Components



Repository data (Gitaly)

Stores Git repositories, branches, commits and tags.



Build artifacts

Generated files from CI/CD pipelines (configurable retention).



Package registry

Stores Maven, npm, PyPI, NuGet and other packages.



Container registry

Stores Docker and OCI images.



Logs

Application, GitLab, Rails, Nginx and CI/CD logs.

3. How GitLab Stores Repository Data (Gitaly)



GitLab
Sends Git
operations
to Gitaly



Gitaly
Stores and
manages Git
repositories



Storage
Data stored on
disk (local or
network storage)

- Gitaly is the component responsible for storing Git repository data.
- It handles all Git operations (clone, fetch, push, merge, etc.).
- Repositories are stored on disk under the Gitaly data directory.
- You can use local storage or external storage such as NFS or cloud storage.

4. Where Data is Stored (Directory Structure)

<code>/var/opt/gitlab/</code>	
<code>gitaly/</code>	→ Git repository data
<code>gitlab-rails/artifacts/</code>	→ CI/CD artifacts
<code>gitlab-rails/packages/</code>	→ Package registry data
<code>gitlab-rails/shared/registry/</code>	→ Container registry data
<code>gitlab-rails/lfs-objects/</code>	→ Git LFS files
<code>gitlab-rails/uploads/</code>	→ Uploads and attachments
<code>gitlab-rails/log/</code>	→ Application logs
<code>gitlab-rails/backups/</code>	→ Backups

5. Manage Storage and Disk Usage



```
# Check disk usage
sudo gitlab-rake gitlab:disk:usage
sudo df -h

# Check repository size
sudo gitlab-rake gitlab-repositories:storage

# Find large projects
sudo gitlab-rake gitlab:projects:storage

# Check artifact size
sudo gitlab-rake gitlab:artifacts:check
```

6. Manage Artifacts and Packages



Artifacts Retention

- Set retention period in project or instance settings.
- Automatically remove old artifacts.
- Keep only the latest successful builds.



Package Registry

- Use retention policies to clean old packages.
- Remove unused package versions.
- Monitor storage usage regularly.



Container Registry

- Enable cleanup policies.
- Remove unused images and tags.
- Use expiring tags and lifecycle rules.

7. Monitor Storage Usage



- Use the GitLab Admin Area → Monitoring → Storage.
- Monitor disk usage for repositories, artifacts, packages and registry.
- Set alerts when disk usage is high.
- Regularly review large projects and cleanup old data.

8. Instance-Level Storage Settings (Admin)



- Configure storage paths for repositories and artifacts.
- Set maximum artifact size and retention period.
- Enable or disable container and package registries.
- Configure object storage (e.g. S3) for large environments.
- Set log rotation and retention settings.
- Monitor Gitaly storage and performance.



Common Mistakes

- Not monitoring disk usage.
- Keeping artifacts forever.
- Storing unused container images.
- Not setting retention policies.
- Allowing projects to grow without limits.
- Ignoring large repositories.

10. Best Practices

- ✓ Set retention policies for artifacts and packages.
- ✓ Use cleanup policies for container images.
- ✓ Monitor disk usage and set alerts.
- ✓ Remove unused data regularly.
- ✓ Use object storage for large data.
- ✓ Keep repositories clean and avoid large files.
- ✓ Review storage usage per project.



11. Your Turn (Practice)

- 1 Check disk usage on your GitLab instance.
- 2 Find the largest projects by storage.
- 3 Set an artifact retention period for a project.
- 4 Enable a cleanup policy for container registry.
- 5 Review and remove old package versions.

Backup creation, configuration and secrets protection, restore testing and recovery.

1. Why Backups are Important?



- Protects your repositories, databases, configurations and CI/CD data.
- Helps you recover from accidental deletion, corruption or server failure.
- Required for disaster recovery and business continuity.
- Allows you to restore to a previous point in time.

2. What is Included in a GitLab Backup?



3. Configure Backups (gitlab.rb)

```
> # Enable and configure backups
gitlab_rails['backup_path'] = "/var/opt/gitlab/backups"
gitlab_rails['backup_keep_time'] = 604800 # 7 days
gitlab_rails['backup_upload_connection'] = {
  'provider' => 'aws',
  'region' => 'us-east-1',
  'aws_access_key_id' => 'YOUR_KEY',
  'aws_secret_access_key' => 'YOUR_SECRET',
  'bucket' => 'gitlab-backups'
}
```

4. Create a Backup

```
> # Create a manual backup
sudo gitlab-rake gitlab:backup:create

# Backup files are stored in
/var/opt/gitlab/backups/

# List backups
ls -lh /var/opt/gitlab/backups/
```

- Backups include Git data, database, artifacts, registry, etc.
- The backup file will be a single .tar file.
- You can schedule automatic backups using cron.

5. Automate Backups (Cron)

```
> # Edit crontab
sudo crontab -e

# Run backup daily at 2 AM
0 2 * * * gitlab-rake gitlab:backup:create
```

6. Protect Backup Files and Secrets



- Store backups in a secure location (e.g. S3, NFS, or offline).
- Use encryption for backup storage.
- Restrict access to backup files.
- Protect AWS or storage credentials using environment variables or a secrets manager.
- Rotate access keys when needed.

7. Restore a Backup (Disaster Recovery)

```
> # Stop GitLab services
sudo gitlab-ctl stop

# Restore from backup file
sudo gitlab-rake gitlab:backup:restore
BACKUP=20240101_020000_17.0.0

# Start services
sudo gitlab-ctl start
```

8. Test Your Restore



- Regularly test backups in a staging environment.
- Verify that projects, users, CI/CD data and registry images are restored correctly.
- Check that runners and integrations work.
- Document the restore process.
- Do not wait for a real disaster to test your backups.

Common Mistakes

- Not enabling regular backups.
- Storing backups only on the same server.
- Not testing the restore process.
- Not protecting backup storage and credentials.
- Keeping backups for too short a period.
- Ignoring artifacts, registry or LFS data.

10. Best Practices

- ✓ Schedule automatic daily backups.
- ✓ Store backups off-site (e.g. S3 or another server).
- ✓ Use encryption and restrict access.
- ✓ Keep multiple backup copies.
- ✓ Set a proper retention period.
- ✓ Test the restore process regularly.
- ✓ Monitor backup success and storage usage.

11. Your Turn (Practice)

- 1 Enable and configure backups in gitlab.rb.
- 2 Create a manual backup.
- 3 Set up a cron job for daily backups.
- 4 Store the backup in a secure location (e.g. S3).
- 5 Test a restore on a non-production environment.

Service health, Prometheus metrics, logs and basic alerting.

1. Why Monitoring and Logging are Important?



GitLab

- Helps you know if GitLab services are healthy and running.
- Shows performance, usage and errors.
- Helps you detect and fix issues early.
- Required for reliability, troubleshooting and capacity planning.
- Supports better user experience and uptime.

2. Key Components to Monitor



GitLab services

web, gitlab-shell, sidekiq, gitaly, postgresql, redis etc.



Prometheus metrics

Collects and exposes metrics from GitLab.



Logs

Application and system logs.



Grafana

Visualize metrics with dashboards.



Alerting

Get notified when problems happen.



System resources

CPU, memory, disk, network and load.

3. Check GitLab Service Health



```
# Check overall GitLab status
sudo gitlab-rake gitlab:check

# Check individual services
sudo gitlab-ctl status

# Check specific service
sudo gitlab-ctl status gitaly

# View detailed health information
sudo gitlab-rake gitlab:env:info
```

4. Prometheus Metrics (GitLab)



```
# GitLab exposes metrics at /-/metrics
curl http://localhost:9168/-/metrics

# Example: check PostgreSQL metrics
curl http://localhost:9168/-/metrics | grep postgresql

# Example: check Sidekiq metrics
curl http://localhost:9168/-/metrics | grep sidekiq
```



- Prometheus collects metrics from GitLab services.
- Metrics include CPU, memory, request rates, job queues, database performance and more.
- Use Grafana to visualize these metrics.

5. View and Manage Logs



```
# View main GitLab log
sudo tail -f /var/log/gitlab/gitlab-rails/production.log

# View nginx log
sudo tail -f /var/log/gitlab/nginx/gitlab_access.log

# View Sidekiq log
sudo tail -f /var/log/gitlab/sidekiq/current

# View Gitaly log
sudo tail -f /var/log/gitlab/gitaly/current

# Search for errors
sudo grep -i error /var/log/gitlab/gitlab-rails/production.log
```

6. Basic Alerting (Example with Prometheus)



```
# Example alert rule (alerts.yml)
groups:
- name: gitlab-alerts
  rules:
  - alert: GitLabInstanceDown
    expr: up{job="gitlab"} == 0
    for: 5m
    labels:
      severity: critical
    annotations:
      summary: "GitLab instance is down"
      description: "GitLab is not responding for 5 minutes"
```

7. Common Metrics to Monitor



- HTTP request rate and response time
- Sidekiq job queue size and processing time
- PostgreSQL connection count and slow queries
- Disk usage and inode usage
- CPU and memory usage
- Git repository storage growth
- Runner job success and failure rate

8. Useful Tools and Endpoints

Tool / Endpoint	Purpose
Prometheus (/-/metrics)	Collects GitLab metrics
Grafana	Visualizes metrics
gitlab-ctl status	Checks service health
/var/log/gitlab/	Application and system logs
node_exporter	System metrics (optional)
alertmanager	Sends alerts (optional)

9. Common Mistakes

- Not monitoring all key services.
- Ignoring disk usage and storage growth.
- Not setting up alerts.
- Not checking logs regularly.
- Leaving default monitoring without customization.

10. Best Practices

- ✓ Monitor all critical services and components.
- ✓ Set up alerts for important failures.
- ✓ Keep disk usage under control.
- ✓ Centralize and analyze logs.
- ✓ Use Grafana dashboards for visibility.
- ✓ Regularly review and test your alerting.

11. Your Turn (Practice)

- 1 Check GitLab service health on your instance.
- 2 View and analyze GitLab logs.
- 3 Access and explore Prometheus metrics.
- 4 Create a simple alert rule for instance down.
- 5 Build a basic Grafana dashboard (optional).

Security updates, token management, network exposure and access auditing.

1. Why Security Administration is Important?



- Protects your source code, CI/CD pipelines and sensitive data.
- Prevents unauthorized access and data leaks.
- Helps you meet compliance and organizational policies.
- Reduces the risk of downtime, breaches and supply chain attacks.
- Keeps your GitLab environment secure and trusted.

2. Key Security Areas in GitLab

Security updates Keep GitLab and dependencies up to date.	Token management Manage PATs, project tokens, group tokens and CI/CD variables.	Network exposure Control access using firewall, reverse proxy and allow lists.	Access auditing Monitor user activity, logins and changes.
Access control Manage users, groups, roles and permissions.	Secrets protection Use masked and protected variables.	Security hardening Disable unnecessary services and features.	Compliance and logging Keep audit logs and review regularly.

3. Security Updates and Version Management

```
> # Check current GitLab version
sudo gitlab-rake gitlab:env:info | grep Version
# Check for available updates (Omnibus)
sudo apt update
sudo apt list gitlab-ee
# Upgrade GitLab (Omnibus)
sudo apt install gitlab-ee=<new_version>
# Reconfigure after upgrade
sudo gitlab-ctl reconfigure
```

4. Token Management

Token Type	Purpose	Key Points
Personal Access Token (PAT)	Access GitLab API on behalf of a user	Set scopes, expiry date and least privilege
Project Access Token	Access a specific project	Use for CI/CD and automation
Group Access Token	Access all projects in a group	Useful for shared automation
Deploy Token	Read-only access to repository/registry	Use for CI/CD and external systems
CI/CD Variable	Store secrets in pipelines	Mark as masked and protected

5. Manage and secure Tokens (CLI and UI)

```
> # Create a personal access token (API)
curl --request POST "https://gitlab.example.com/api/v4/user/personal_access_tokens" \
--header "PRIVATE-TOKEN: <YOUR_ADMIN_TOKEN>" \
--data "name=automation" \
--data "scopes[]=api" \
--data "expires_at=2026-12-31"
```

- Use minimal required scopes.
- Set an expiry date for all tokens.
- Rotate tokens regularly.
- Revoke unused or suspicious tokens.
- Use masked and protected CI/CD variables for secrets.

6. Control Network Exposure



- Use a reverse proxy (Nginx) with HTTPS.
- Restrict access to trusted IP addresses.
- Disable unused services (e.g. registry, pages).
- Use a firewall and security groups.
- Enforce strong TLS configuration and certificates.

7. Access Auditing and Monitoring



- Enable and review audit logs.
- Monitor user logins and failed login attempts.
- Track changes to projects, groups and settings.
- Use external log management (e.g. ELK, Loki, Splunk).
- Set up alerts for suspicious activity.

8. Useful Security Commands

Command	Purpose
<code>sudo gitlab-ctl status</code>	Check service status
<code>sudo gitlab-rake gitlab:env:info</code>	View configuration and version
<code>sudo gitlab-rake audit:enable</code>	Enable audit events (if available)
<code>sudo gitlab-ctl tail</code>	View real time logs
<code>sudo netstat -tulnp grep gitlab</code>	Check open ports and services

9. Common Mistakes

- Not applying security updates.
- Using tokens without expiry dates.
- Giving excessive permissions.
- Exposing GitLab to the internet without protection.
- Not monitoring audit logs.
- Storing secrets in plain text.

10. Best Practices

- Keep GitLab and dependencies up to date.
- Use least privilege access.
- Rotate tokens and review access regularly.
- Restrict network exposure.
- Enable audit logs and monitor activity.
- Use masked and protected variables for secrets.
- Regularly review users, groups and permissions.

11. Your Turn (Practice)

- Check your current GitLab version.
- Create a personal access token with limited scope.
- Set a masked and protected CI/CD variable.
- Restrict network access to trusted IPs.
- Enable and review audit logs.

Upgrade paths, compatibility checks, maintenance and rollback planning.

1. Why Upgrades and Maintenance are Important?



- Keeps your GitLab instance secure with the latest patches.
- Provides new features, performance improvements and bug fixes.
- Ensures compatibility with dependencies (PostgreSQL, Redis, Gitaly, etc).
- Reduces the risk of downtime and security vulnerabilities.
- Helps you maintain a stable and supported environment.

2. Upgrade Paths and Release Types

Patch Release



Security fixes and critical bug fixes.
Recommended to apply quickly.

Minor Release



New features, improvements and bug fixes.
Usually backward compatible.

Major Release



Significant new features and changes.
Check compatibility and plan carefully.



3. Check Current Version and Upgrade Path

```
> # Check current GitLab version
sudo gitlab-rake gitlab:env:info | grep "GitLab"

# Check if a new version is available
sudo gitlab-rake gitlab:check | grep Upgrade

# View detailed version information
cat /opt/gitlab/embedded/service/gitlab-rails/REVISION
```

5. Perform the Upgrade (Omnibus Example)

```
> # Backup before upgrade
sudo gitlab-rake gitlab:backup:create

# Update package repository
sudo apt update

# Install the new version (example: 18.0.0)
sudo apt install gitlab-ee=18.0.0-ee.0

# Reconfigure GitLab
sudo gitlab-ctl reconfigure

# Verify the new version
sudo gitlab-rake gitlab:env:info | grep "GitLab"
```

8. Verify After Upgrade



- Confirm GitLab version and all services are running.
- Check application logs for errors.
- Verify user authentication, CI/CD pipelines and key features.
- Monitor system resources (CPU, memory, disk).
- Ensure integrations and custom configurations are working.

9. Common Mistakes

- Skipping backup before upgrade.
- Not checking dependency compatibility.
- Upgrading directly in production without testing.
- Ignoring custom scripts and integrations.
- Not monitoring after the upgrade.

10. Best Practices



- ✓ Follow the official GitLab upgrade path.
- ✓ Always test in a staging environment.
- ✓ Take backups and verify recovery.
- ✓ Keep documentation of the process.
- ✓ Schedule upgrades during low traffic.
- ✓ Monitor services and logs after upgrade.

11. Your Turn (Practice)

- 1 Check your current GitLab version.
- 2 Verify if an upgrade is available.
- 3 Review system requirements.
- 4 Perform a test upgrade in a staging environment.
- 5 Plan a rollback and test it.
- 6 Monitor and verify the upgrade results.

4. Pre-Upgrade Compatibility Checks



Check Dependencies

- Verify PostgreSQL, Redis, Gitaly and other components are compatible with the target version.



Check System Requirements

- Review OS, disk space, memory, CPU and filesystem requirements.



Check Customizations

- Ensure custom scripts, integrations and third-party tools are compatible.



Test in a Staging Environment

- Always test the upgrade process in a non-production environment first.

6. Maintenance Tasks



- Schedule maintenance window and notify users.
- Put instance in read-only mode if required.
- Cleanup old packages and unnecessary files.
- Optimize database and run housekeeping tasks.
- Verify services are healthy after upgrade.
- Monitor logs and system performance.

7. Rollback Planning



- Always take a full backup before upgrading.
- Keep the previous package version available.
- Document rollback steps and test them.
- Verify data integrity after rollback.
- Have a clear communication plan.

HTTP 502, failed pipelines, stuck runners, storage exhaustion and service failures.

1. Common GitLab Failures and Symptoms

	HTTP 502	- Cannot access GitLab UI. 502 Bad Gateway error. - Usually caused by Nginx, Puma or backend services.
	Failed Pipelines	- Pipelines fail or get stuck. - Jobs timeout or error. - Usually caused by runner issues, network, or misconfiguration.
	Stuck Runners	- Jobs remain in pending state. - Runners offline or not picking jobs. - Usually caused by runner service, tags or resource issues.
	Storage Exhaustion	- GitLab becomes slow or unavailable. - Disk space 100%. - Repositories, artifacts and logs consume too much space.
	Service Failures	- GitLab services not running. - UI, CI/CD, registry or database issues. - Usually caused by configuration, resource or dependency failures.

2. Troubleshoot HTTP 502 (Nginx / Puma)

```
# Check GitLab service status
sudo gitlab-rake gitlab:check

# Check Nginx status and logs
sudo gitlab-ctl status nginx
sudo tail -f /var/log/gitlab/nginx/error.log

# Check Puma status and logs
sudo gitlab-ctl status puma
sudo tail -f /var/log/gitlab/puma/puma.stderr.log
```



- ✓ Check if Nginx can reach Puma.
- ✓ Verify Puma workers are running.
- Check system resources (CPU, memory, disk).
- Review firewalls and network configuration.

3. Troubleshoot Failed Pipelines

```
# Check pipeline and job details
sudo gitlab-rails runner "Ci::Job.find(<JOB_ID>)"

# Check runner status
sudo gitlab-rails runner "Ci::Runner.all"

# View pipeline logs
sudo gitlab-rails runner "puts '\n\n' +  
Ci::Job.find(<JOB_ID>).trace"
```

- Check job logs for errors.
- Verify runner is online and has correct tags.
- Check CI/CD variables and secrets.
- Look for network, permission or resource issues.

4. Troubleshoot Stuck Runners

```
# Check runner status
sudo gitlab-runner status

# List running runners
sudo gitlab-rails runner "Ci::Runner.all.pluck(:id,  
:description, :online, :paused)"

# Restart runner service (if self-hosted)
sudo systemctl restart gitlab-runner
```

- Verify runner is online and not paused.
- Check runner logs: `/var/log/gitlab-runner/`.
- Ensure correct tags and permissions.
- Check for resource issues (CPU, memory, disk).
- Re-register runner if necessary.

5. Troubleshoot Storage Exhaustion

```
# Check disk usage
df -h

# Find large GitLab directories
sudo du -sh /var/opt/gitlab/* | sort -h

# Check repository and artifacts size
sudo gitlab-rake gitlab:usage:check
```

- Identify large repositories, artifacts and logs.
- Clean up old artifacts and container images.
- Set up retention policies.
- Move data to external storage if needed.
- Monitor disk usage regularly.

6. Troubleshoot Service Failures

```
# Check all GitLab services
sudo gitlab-ctl status

# Reconfigure and restart services
sudo gitlab-ctl reconfigure

# View specific service logs
sudo gitlab-ctl tail <service-name>

# Example: Rails, Sidekiq, Gitaly, Redis, PostgreSQL
sudo gitlab-ctl tail gitaly
```

- Check service status and logs.
- Verify dependencies (PostgreSQL, Redis, Gitaly).
- Ensure sufficient system resources.
- Check configuration changes.
- Restart or reconfigure services.

7. Common Mistakes

- Ignoring logs and error messages.
- Not checking system resources.
- Not verifying service dependencies.
- Ignoring runner and pipeline issues.
- Letting disk space reach 100%.

8. Best Practices

- Monitor services, logs and resources.
- Set up alerts for failures and disk usage.
- Use retention policies for artifacts and logs.
- Keep GitLab and dependencies updated.
- Document troubleshooting steps.
- Test recovery procedures regularly.

9. Your Turn (Practice)

- Diagnose and fix an HTTP 502 error.
- Investigate a failed pipeline and find the root cause.
- Fix a stuck runner.
- Identify and resolve storage exhaustion.
- Troubleshoot a specific service failure.

Deploy GitLab, configure access, register a runner, execute a pipeline, create a backup and verify recovery.

1. Deploy GitLab (Self-Managed)



- ✓ Deploy a GitLab instance using Omnibus on a Linux server.
- ✓ Access the GitLab web UI.
- ✓ Verify all services are running.
- ✓ Configure initial settings (hostname, external URL).

```
> # Add GitLab repository and install
curl -fsSL https://packages.gitlab.com/
gitlab/gitlab-ee/gpgkey | sudo gpg --dearmor \
-o /usr/share/keyrings/gitlab-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/gitlab-
archive-keyring.gpg] https://packages.gitlab.com/
gitlab/gitlab-ee/ubuntu focal main" | sudo tee
/etc/apt/sources.list.d/gitlab.list
sudo apt update
sudo apt install gitlab-ee -y
```

3. Register a GitLab Runner



- ✓ Install and register a runner.
- ✓ Use a specific executor (docker or shell).
- ✓ Verify runner is online.
- ✓ Assign runner to a project or group.

```
> # Install GitLab Runner
curl -L https://packages.gitlab.com/gitlab-runner/gitlab-
runner/packages/ubuntu/focal/gitlab-runner_amd64.deb -o
gitlab-runner.deb
sudo dpkg -i gitlab-runner.deb

# Register runner (interactive)
sudo gitlab-runner register

# Check runner status
sudo gitlab-runner status
```

5. Create a Backup



- ✓ Create a full GitLab backup.
- ✓ Verify backup file and contents.
- ✓ Store backup in a secure location.
- ✓ Automate backups using cron (optional).

```
> # Create a backup
sudo gitlab-rake gitlab:backup:create

# List backups
ls -lh /var/opt/gitlab/backups/

# Example: copy to external storage
sudo rsync -av /var/opt/gitlab/backups/ \
user@backup-server:/backup/gitlab/
```

7. Common Mistakes

- Using the wrong GitLab version.
- Not configuring external URL correctly.
- Runner not picking up jobs.
- Not verifying backup file.
- Skipping restore testing.
- Incorrect permissions and access control.

8. Best Practices



- ✓ Keep GitLab updated and secure.
- ✓ Use dedicated runners for production.
- ✓ Implement regular backups.
- ✓ Test restore process periodically.
- ✓ Use least privilege access control.
- ✓ Monitor services, logs and storage.
- ✓ Document all configurations.

2. Configure Access and Users



- ✓ Access the GitLab web UI.
- ✓ Create users and groups.
- ✓ Set roles and permissions.
- ✓ Configure authentication (local or LDAP/OAuth).
- ✓ Configure project and group settings.

```
> # Example: Create a group and add a user (GitLab API)
curl --header "PRIVATE-TOKEN: YOUR_TOKEN" \
--data "name=devops-team" \
"https://gitlab.example.com/api/v4/groups"

# Add user to group
curl --request POST --header "PRIVATE-TOKEN: YOUR_TOKEN" \
--data "user_id=123&access_level=30" \
"https://gitlab.example.com/api/v4/groups/456/members"
```

4. Execute a Pipeline



- ✓ Create a project and add a .gitlab-ci.yml file.
- ✓ Run a pipeline using the registered runner.
- ✓ Monitor job execution and logs.
- ✓ Verify successful build and deployment steps.

```
> # Example .gitlab-ci.yml
stages:
  - build
  - test
build_job:
  stage: build
  script:
    - echo "Building the application"
    - echo "Build completed"
test_job:
  stage: test
  script:
    - echo "Running tests"
    - echo "Tests completed"
```

6. Verify Recovery



- ✓ Restore GitLab from a backup.
- ✓ Verify services are running.
- ✓ Confirm projects, users and data are restored.
- ✓ Test pipeline execution after recovery.

```
> # Stop GitLab services
sudo gitlab-ctl stop

# Restore from backup file
sudo gitlab-rake gitlab:backup:restore \
BACKUP=YYYYMMDD_HHMMSS_gitlab_backup.tar

# Start services
sudo gitlab-ctl start

# Verify
sudo gitlab-ctl status
```

9. Your Turn (Practice)



- 1 Deploy a GitLab instance.
- 2 Configure users and groups.
- 3 Register a runner.
- 4 Create and run a pipeline.
- 5 Create a backup.
- 6 Restore and verify your GitLab instance.